Assignment Predictive Process Monitoring

Process Mining

JM0211-M-6

October 28, 2026

Group 9

M.W. van Donkelaar - *2155114*

J.G. Duin - *2146737*

C.N. Hiddink - *2156559*

F. Molenaar - *2155354*

1 Motivation and Scientific Background

1.1 Motivation

Predictive Process Monitoring (PPM) is about predicting what is likely to happen next in a process, such as its next activity, remaining time, or final outcome. For example, in a hospital, early outcome prediction helps plan capacity and prioritize patients before delays occur. Over the past few years, there have been various improvements in PPM, particularly with Large Language Models (LLMs) leveraging their embedded knowledge to generate accurate predictions while demonstrating robustness in handling noisy and unstructured data (Requeima et al., 2024).

Building on these advances, this research combines two papers with the goal of evaluating whether LLMs can improve the performance of traditional machine learning techniques on (clustered) traces. Francescomarino et al. (2019) address outcome prediction by clustering historical traces, by assigning a running trace to the most similar cluster and training one classifier per cluster. In contrast, Padella et al. (2025) train no model. Instead, they use a prompt to let an LLM predict the total time of a running case. However, their method was only tested for remaining-time prediction. Therefore, in this research, their prompt is adapted for outcome prediction and evaluated across the four labeling strategies of Francescomarino et al. (2019) using BPIC 2011 data.

1.2 Embedding in Course and Relation to Other Projects

PPM is one of the techniques learned in the classes of this course and forms the basis for prescriptive process optimisation, which selects actions based on predictions. The project also covers the learning outcomes of critically evaluating new algorithms and applying state-of-the-art methods to real-world data. It also relates process mining to data science.

Our topic is closely related to those of the other groups, especially those in the field of PPM, though all topics are connected through the broader context of process mining. For the PPM assignments, the same setup structure (pipeline) is used, but different elements are changed; in our project, we changed the learner. Other groups changed the feature space by adding information on other cases, and another group focused on post-hoc explainability for predictions.

2 Conceptual Approach

This chapter outlines the conceptual design (Figure 1) of the predictive process monitoring (PPM) framework just introduced. While Francescomarino et al. (2019) address outcome prediction by clustering historical traces, assigning a running trace to the most similar cluster and training one classifier per cluster, Padella et al. (2025) bypass traditional training by letting an LLM predict the total time of a running case. Because Padella et al. (2025) evaluated their approach on total time prediction, this approach adapts their prompt engineering strategy for binary outcome prediction.

2.1 Input

The approach requires an event log structured as a sequence of timestamped events per trace, capturing a control-flow perspective with activity execution sequences, a data perspective comprising event- and case-level attributes such as age, diagnosis, and treatment code in BPIC 2011, and a labeling function where each completed historical case is classified as normal (negative) or deviant (positive) using Linear Temporal Logic (LTL) rules ranging from φ_1 to φ_4. Furthermore, an 80–20 chronological split divides the log into historical training traces and test traces.

2.2 Set Parameters

Several parameters are used in this approach. First, to determine the length of the prefixes, a prefix gap step size of $g = 5$, corresponding to prefix lengths up to 21, is used, as in the paper. For clustering, the number of clusters k is evaluated across a range from 15 to 35 with a step of 5 and selected via the Bayesian Information Criterion for model-based clustering, alongside parameters of minPoints $= 4$ and $\epsilon = 0.125$ for DBSCAN. Every prefix receives the most likely class, a decision threshold of 0.5.

2.3 Method

Two predictive workflows are evaluated. The first follows a traditional clustering-based PPM framework. Two clustering techniques are considered: model-based clustering and DBSCAN. For each resulting cluster, a separate classifier (Decision Tree or a Random Forest) is trained, with each prefix represented by its last event (last-state encoding). At prediction time, a new prefix is assigned to its nearest cluster, and the classifier associated with that cluster predicts the outcome. In addition to the paper, they are compared against an unclustered Random Forest baseline model.

The second workflow is the LLM-based PPM. The traces are formatted into strings via sequential encoding containing global attributes and activity sequences. The prompt of Padella et al. (2025) is adjusted to the correct context and prediction task. The prompt is tested with the LLM Qwen2.5 (3B) in three versions, with context, without context, and with the control flow only. As an additional comparison, the same input is given to JEV, which answers a yes/no question.

2.4 Output

For any running case, the framework produces a predicted binary outcome classifying the trace as normal or deviant relative to the LTL rule, alongside evaluation metrics tracking the F1-score and accuracy.

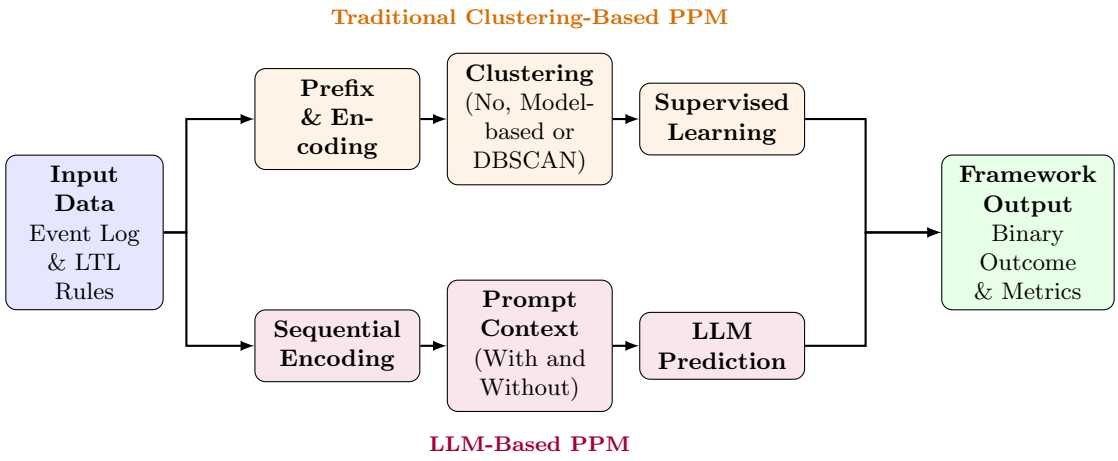


Figure 1: Conceptual overview of the two predictive process monitoring workflows.

3 Implementation

Our code is implemented in Python 3.14.0. Dependencies are managed with `uv` and the `pyproject.toml` lists all the packages. Table 1 lists the main packages used in the code. The code consists of three modules: `model_pipeline.py` for data preparation, prefix extraction, clustering, classification and scoring, `llm_pipeline.py` calls the LLM and `jev_pipeline.py` calls the Jev API. We used one notebook per rule (`Rule1.ipynb` to `Rule4.ipynb`), each notebook calls these modules individually.

Table 1: Main packages and their use.

Package	Version	Used for
pm4py	2.7.23.8	Reading the XES event log
pandas	3.0.6	Data handling
scikit-learn	1.9.1	GMM, DBSCAN, DT, RF, encoding, F1, stratified sampling
rapidfuzz	3.14.6	Normalised Levenshtein distances
openai	3.31.0	Client for the Ollama server
typesafe-sdk	0.7.4	Client for the JEV API
matplotlib	3.11.2	Figures

3.1 Data preparation

The BPIC 2011 log consists of 1,143 cases and is read with `pm4py.read_xes` into a dataframe. Timestamps are converted to Amsterdam time and then grouped per case. Each rule has a dedicated Python function that checks whether a completed case satisfies the rule. Each case is labelled 1 when it breaks the rule and 0 if not.

The cases use an 80/20 split by start time: the oldest 914 cases are used for training, the newest 229 for testing. Each case is then cut into prefixes of length 1, 6, 11, 16 and 21 events. This results in 3,401 training and 1,007 test prefixes per rule.

The classifiers use the attributes of the last event of each prefix as features, as in Francescomarino et al. (2019): org:group, Number of executions, Specialism code, Producer code, Section and Activity code, and the case attributes Age, Diagnosis, Diagnosis code and Treatment code, which are the same for every event of a case. All attributes are treated as categories, with missing values as a category of their own. They are converted to numbers with scikit-learn's `OrdinalEncoder`, values not seen during training become -1.

3.2 Clustering and classification

3.2.1 Reproduction of the paper

Following Francescomarino et al. (2019), prefixes are clustered on control flow only, and one classifier is trained per cluster on the features from Section 3.1. Because the labels are not used, the clusters are the same for all rules.

Model-based clustering fits a Gaussian mixture on the frequency encoding: one dimension per activity, holding its count divided by the prefix length. k ranges over 15, 20, 25, 30, 35 and is selected by the lowest BIC. We use a diagonal instead of a full covariance matrix: with one dimension per activity, a full matrix needs about 194,000 parameters per cluster and becomes singular. A test prefix goes to its most likely component.

DBSCAN uses the normalised Levenshtein distance between activity sequences, with $\epsilon = 0.125$ and minPoints = 4 as in the paper. Noise points form one extra group. A test prefix takes the cluster of the nearest training prefix. Each cluster gets a decision tree or a random forest with 100 trees (scikit-learn defaults, seed 42, no tuning). A cluster with only one class gets no classifier and receives the most common training label. Predictions use a threshold of 0.5, without the paper's minimum class probability or support, so every prefix gets a prediction.

In addition, a baseline Random Forest model without clustering was trained, to measure the effectiveness of the clustering.

3.2.2 Own additions

Two models without case attributes are added to the reproduction. The paper uses all attributes of the log, including Age, Diagnosis, Diagnosis code and Treatment code. In the XES file these are trace-level attributes without a timestamp, and pm4py copies them onto every event. As a result, even a prefix of length 1 carries the diagnoses and treatment codes of the complete case, including those registered later in the treatment. The log does not record when a diagnosis was added, so information known at intake cannot be separated from information that only becomes available later. In this variant, the four case attributes are removed and only the event attributes are kept. This measures how much of the score depends on those attributes instead of the sequence and the cluster. It is a similar ablation as the control-flow-only version of the LLM.

The paper encodes a prefix by the attributes of its last event. This assumes that the last event also holds the values of all earlier activities (global data). In this log, event attributes are not carried forward, so the classifier never sees which activities happened earlier in the case: the control flow only decides which cluster a prefix falls in. The second variant adds the history of the prefix as a boolean encoding. There is one column per activity in the training prefixes, set to 1 if that activity has occurred in the prefix. The rules are LTL formulas about whether activities occur, so "has X occurred yet" is exactly the state they depend on. This is the presence version of the paper's own frequency encoding. With about 900 training cases, one column per activity can be learned, whereas one column per position in the prefix could not. The encoding loses the order of activities, which matters for rules 2 and 3, which depend on order. Everything in the encoding is known at the time of the prefix, so no

future information leaks in. This variant builds on the previous one, so the case attributes stay out. Comparing it with the variant without case attributes shows what the history adds.

3.3 LLM-based prediction

The LLM is `qwen2.5:3b`, a 3.1B parameter 4-bit quantised model that runs with Ollama on a remote server and is called through the `openai` Python package. If a prompt is longer than the context window Ollama silently cuts off the start which is where the instructions are. We therefore set the context window to 32,768 tokens and check before each run that the server uses it. Prompts that may not fit stop the run with an error. Each request uses temperature 0, so the same prompt gives the same answer with at most 800 tokens for the answer, a 120-second timeout and up to three retries.

The prompt template (Appendix A) has five placeholders, which are filled in for each prediction: the rule in words (`$rule`), an explanation of the age and diagnosis keys (`$attributes`), background on the process (`$domain`), six example cases (`$examples`), and the test prefix to predict (`$case`). Only the test prefix changes between predictions.

Each case is written as one line of JSON with the patient's age, the list of diagnoses and `ActTimeSeq`: the activities in order, each with the number of days since the case started. The example cases also include their outcome. The case to predict ends with `["Running"]`, to show that it is not finished. The six examples are complete cases from the training set, drawn so that the share of both labels matches the training set using stratified sampling and a seed of 42. The same six examples are used for every prediction of a rule.

The prediction is the 0 or 1 after `[[ ## answer ## ]]` in the reply. If no answer can be read, the most common label of the training set is used. This happened for 115 of the 12,084 answers, almost always because the model was still reasoning when it reached the 800 token limit.

We run three versions on all 1,007 test prefixes per rule: the full prompt, full prompt without context where `$attributes` and `$domain` are left empty and control flow only where age and diagnoses are removed form all cases.

3.4 JEV

JEV is a model by TypeSafe that answers typed questions instead of writing free text. We call it through the `typesafe-sdk` package, fixed to version jev-1.13.0 so that reruns use the same model. JEV gets the same input as the LLM: the rule, the six example cases and the case to predict. It is asked one yes/no question: "Will the rule hold for the running case once it has completed?". JEV returns the probability that the rule holds; below 0.5 we predict 1. Like the LLM, JEV runs with and without context.

4 Evaluation Design and Results

We use the test set of 1,007 prefixes per rule described in Section 3.1. As in the original study, a case that violates the rule is labelled deviant (1) and otherwise normal (0). This label is the ground truth. It is determined on the completed case, and every prefix of that case receives the same label. Our class distribution matches the one reported by Francescomarino et al. (2019) to within a few cases, so our labelling reproduces theirs. The classes are unbalanced and differ per rule: 47%, 38%, 61%, and 56% of the test prefixes are deviant for rules 1 to 4. No artificial data was used.

The main metric is the F1 score of the deviant class, as in the paper. Accuracy is used as the secondary metric. Both are computed over all test prefixes and per prefix length.

4.1 Reproduction & LLM-based Prediction

Table 2 shows that rule 3 is the easiest to predict (F1 score up to 0.849) and rule 1 the hardest (F1 scores between 0.429 and 0.531). The random forest without clustering scores best or near-best on the F1 scores and has the highest accuracy on all four rules. Figure 2 in Appendix B shows the F1 score per prefix length. For rules 2–4 the scores stay fairly stable over the prefix lengths. For rule 1, they drop after the first prefix and rise again at length 21, except for the model without clustering.

The LLM scores lower F1 scores than the classifiers on rules 2–4. On rule 1 its F1 score is higher (0.500–0.579), but its accuracy (0.492–0.505) is the lowest of all models. Across all rules its accuracy stays below the share of the largest class. On rule 1 its higher F1 score therefore comes from often predicting deviant, not from being accurate. Leaving out the context has no consistent effect, and the control flow version gives the highest F1 score on rules 1–3, so the LLM makes little use of the age and diagnoses. JEV is the best model on rule 1 (F1 score 0.669, accuracy 0.686), but on rule 2 it predicts almost every case as normal.

Table 2: F1 score and accuracy per rule for the reproduced models, the LLM and JEV ($n = 1007$ per rule).

	Rule 1		Rule 2		Rule 3		Rule 4	
Model	F1	Acc	F1	Acc	F1	Acc	F1	Acc
M-based + DT	0.470	0.604	0.628	0.708	0.817	0.764	0.737	0.707
M-based + RF	0.429	0.606	0.668	0.756	0.842	0.787	0.765	0.744
DBSCAN + DT	0.531	0.608	0.619	0.692	0.785	0.720	0.728	0.709
DBSCAN + RF	0.456	0.609	0.650	0.746	0.834	0.776	0.778	0.756
No clustering + RF	0.473	0.620	0.655	0.762	0.849	0.795	0.793	0.770
LLM								
LLM	0.556	0.492	0.292	0.562	0.507	0.497	0.574	0.499
LLM (no context)	0.500	0.505	0.187	0.552	0.579	0.521	0.597	0.502
LLM (control flow)	0.579	0.503	0.368	0.570	0.589	0.512	0.553	0.514
JEV								
JEV	0.584	0.671	0.150	0.650	0.609	0.598	0.704	0.549
JEV (no context)	0.669	0.686	0.150	0.650	0.657	0.616	0.710	0.556

4.2 Additional ML models

Table 3 and the Figure 3 in Appendix C show that removing the case attributes lowers the F1 score of every model on every rule. The drop is largest for rule 2, where F1 falls from 0.62–0.67 to 0.11–0.25. Adding the prefix history recovers part of this for rules 1 and 2: rule 2 rises to an F1 of 0.36-–0.47. Rule 3 stays about the same and rule 4 drops slightly.

Table 3: Results of own additions. F1 score and accuracy per rule ($n = 1007$ per rule). Results of the original setup are given in Table 2.

	Rule 1		Rule 2		Rule 3		Rule 4	
Model	F1	Acc	F1	Acc	F1	Acc	F1	Acc
Without case attributes								
M-based + DT	0.272	0.547	0.117	0.627	0.754	0.619	0.589	0.554
M-based + RF	0.330	0.544	0.180	0.620	0.744	0.608	0.636	0.570
DBSCAN + DT	0.295	0.558	0.139	0.632	0.699	0.584	0.585	0.552
DBSCAN + RF	0.358	0.558	0.247	0.637	0.729	0.596	0.586	0.550
No clustering + RF	0.384	0.557	0.113	0.641	0.762	0.619	0.656	0.548
History, without case attributes								
M-based + DT	0.379	0.596	0.387	0.642	0.759	0.641	0.540	0.514
M-based + RF	0.408	0.576	0.437	0.632	0.759	0.628	0.563	0.527
DBSCAN + DT	0.406	0.587	0.355	0.640	0.750	0.638	0.573	0.534
DBSCAN + RF	0.428	0.581	0.417	0.653	0.770	0.648	0.566	0.531
No clustering + RF	0.395	0.584	0.469	0.662	0.759	0.625	0.581	0.542

4.3 Discussion

In our setting, the LLM-based approaches do not improve on the traditional models, which was the main question of this research. A likely reason is that the outcome of each rule is decided by activities that often occur after the prefix. A classifier can learn statistical patterns from 3,401 training prefixes, whereas the LLM only sees the rule and six examples. Knowing the rule does not help it predict activities that have not happened yet. The small size of the model probably adds to this.

The case attributes explain most of the classifiers' performance. Without them, accuracy is close to the share of the largest class (0.53, 0.62, 0.61 and 0.56 for rules 1 to 4). These attributes describe the whole case, including diagnoses registered later. Part of the score in the original setup, and probably also in the paper, therefore rests on information that is not yet available at prediction time. This also affects the comparison with the LLM, which only receives Age and Diagnosis. The prefix history recovers part of the gap, but not all of it, which suggests that the control flow of the first 21 events is only weakly predictive of the outcome.

The paper presents clustering as an improvement, but in our setup it gives no consistent gain. Two factors may explain this. First, splitting 3,401 prefixes over 15 to 35 clusters leaves each classifier with far less training data. Second, the paper measures its gains with a probability threshold, while we predict every prefix. Finally, because the classes are imbalanced, a model that often predicts deviant gets a high F1 score without being accurate, as the LLM on rule 1 shows. F1 should therefore be read together with accuracy.

5 Limitations and Future Work

Our evaluation predicts every prefix without a minimum class probability, so it does not measure the failure rate and earliness on which Francescomarino et al. (2019) base the benefit of clustering. All results come from one event log, one temporal split and one run, without significance tests, so small differences between models may not be meaningful. Because the log does not record when case attributes such as diagnoses were registered, they cannot be used without including information from later in the case. For the LLM, we use one small model, the same example cases for every prediction, and the LLM gives no class probability.

Future work could obtain class probabilities from the LLM, for example through token probabilities, so that the threshold-based evaluation of the paper can be applied to all models. Selecting the example cases per prediction, for instance from the cluster of the running case, would give the LLM more relevant examples and combine both approaches.

References

Francescomarino, C. D., Dumas, M., Maggi, F. M., and Teinemaa, I. (2019). Clustering-Based Predictive Process Monitoring . *IEEE Transactions on Services Computing*, 12(06):896–909.

Padella, A., Frazzetto, P., Navarin, N., and de Leoni, M. (2025). *Enhancing Predictive Process Monitoring on Small-Scale Event Logs Using LLMs*, pages 274–290. Springer, Cham.

Requeima, J., Bronskill, J., Choi, D., Turner, R. E., and Duvenaud, D. (2024). Llm processes: Numerical predictive distributions conditioned on natural language.

A Prompt

```
You are an expert in process mining and machine learning. Your task is to predict the 'outcome' of
process instances based on event logs, as each process instance is a sequence of activities.

An event log is a collection of traces, where each trace represents a process instance.
Each trace is mapped as a sequence of activities and integers representing the days since the start
of the case.
The log is represented as a python list containing one dictionary for each trace. Included in it are:
- the key ActTimeSeq, which value is a list of [activity, cumulative elapsed days since the start of
the case]
- the key outcome, which value is 0 or 1, that is the value to predict. The outcome refers to the
complete case.
$attributes

The outcome of a completed case is determined as follows:
$rule

All interactions will be structured in the following way, with the appropriate values filled in.

[[ ## reasoning ## ]]
{your step-by-step reasoning, at most 5 short steps}

[[ ## answer ## ]]
{only the digit 1 or 0, nothing else}

[[ ## completed ## ]]

In adhering to this structure, your objective is to analyze the event log, and apply reasoning to
predict the outcome for a new case. This case belongs to a not-yet-completed process instance,
represented by the label Running in ActTimeSeq, indicating that more activities are expected before
reaching the conclusion of the process instance.

Ensure to articulate each step of your thought process in the reasoning field, detailing how you
identify relationships with past cases and leverage your intuition about the meaning of activities
to arrive at the solution. The answer should be the final prediction of the outcome for the given
process instance.
Respond with the corresponding output fields, starting with the field [[ ## reasoning ## ]], then [[
## answer ## ]], and then ending with the marker for [[ ## completed ## ]].

Your task is to learn from them and predict the 'outcome' values for that traces.

$domain

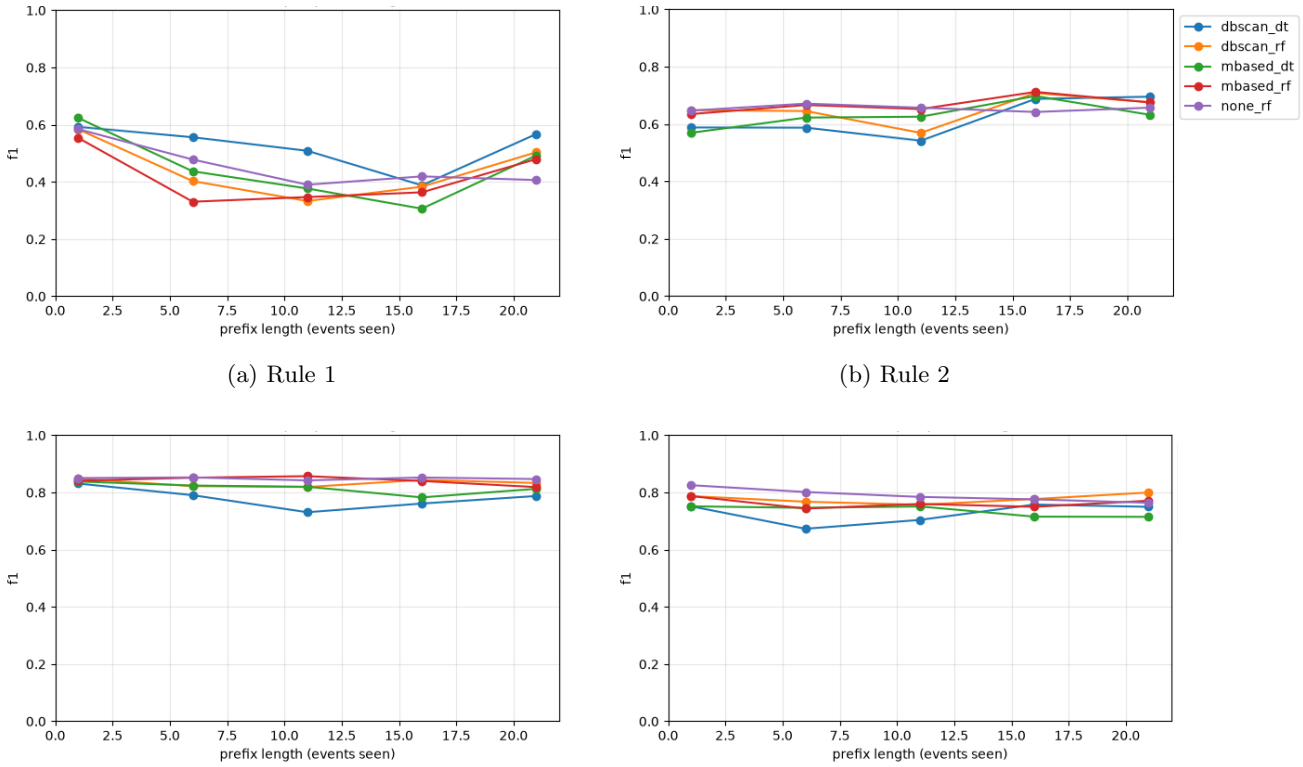
The following list shows the first activities of some completed example cases with the outcome of
the completed case. The activities that decide the outcome may occur after the activities shown.

$examples

Now predict the outcome for this new uncompleted case, considering that the case is still running:

$case
```

B Reproduction Paper



(a) Rule 1

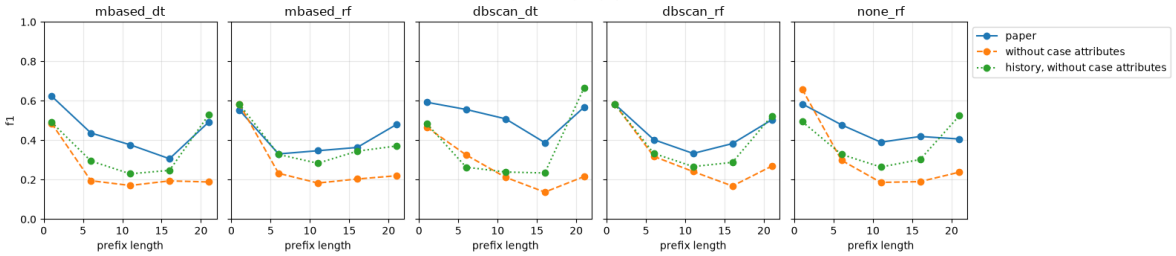
(b) Rule 2

(c) Rule 3

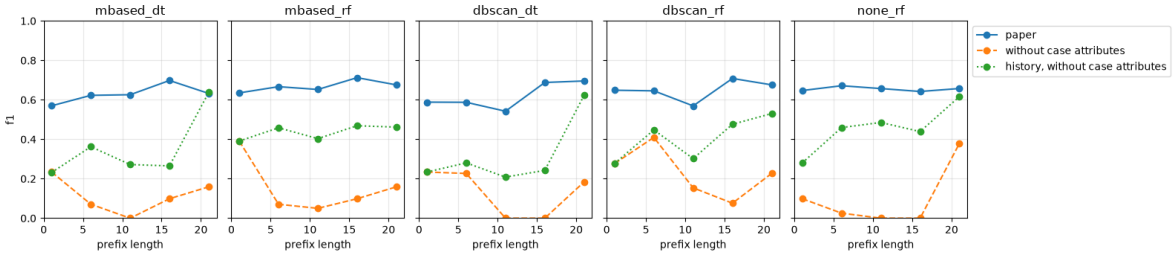
(d) Rule 4

Figure 2: Reproduction of the results of Francescomarino et al. (2019). F1 score per prefix length for rules 1-4, with case attributes included.

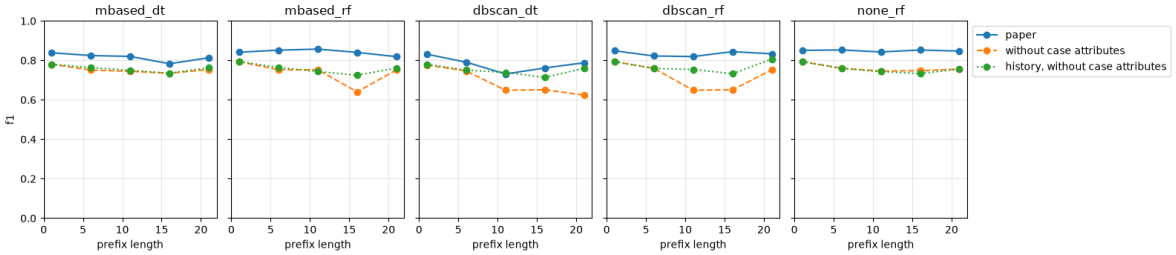
C Own Additions



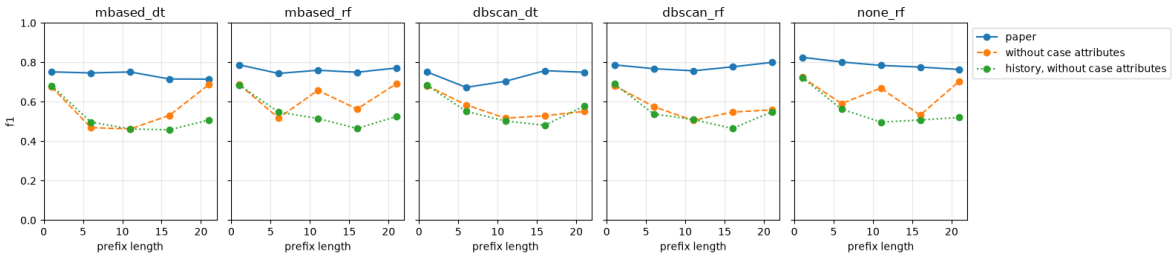
(a) Rule 1



(b) Rule 2



(c) Rule 3



(d) Rule 4

Figure 3: F1 score per prefix length for each configuration and rules 1-4. Original setup with case attributes (solid), without case attributes (dashed), and with history features but without case attributes (dotted).

D Technology Statement

During the preparation of this work, We used Claude/Gemini in order to assist with calculations, visualisations, code suggestions, editing, and brainstorming. The following parts of the assignment were affected/generated by AI tool usage: All. After using this tool/service, **Max van Donkelaar**, **Jelles Duin**, **Coen Hiddink** and **Fleur Molenaar** evaluated the validity of the tool's outputs, including the sources that generative AI tools have used, and edited the content as needed. As a consequence, **Max van Donkelaar**, **Jelles Duin**, **Coen Hiddink** and **Fleur Molenaar** take full responsibility for the content of their work.